```
import pandas as pd
import numpy as np
```

# Load dataset

```
df = pd.read_csv("train.csv")
```

# Display dataset

```
print(df.head())
```

# Check missing values

```
print(df.isnull().sum())
```

# Fill missing values

```
df['Age'].fillna(df['Age'].mean(), inplace=True)
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
```

# Drop Cabin column

```
df.drop('Cabin', axis=1, inplace=True)
```

# Remove unnecessary columns

df.drop([‘Name’, ‘Ticket’, ‘PassengerId’], axis=1, inplace=True)

# Convert categorical to numeric

df[‘Sex’] = df[‘Sex’].map({‘male’:0, ‘female’:1})
df[‘Embarked’] = df[‘Embarked’].map({‘S’:0, ‘C’:1, ‘Q’:2})

# Final null check

print(df.isnull().sum())

# Display processed data

print(df.head())

# Save cleaned dataset

df.to_csv(“Titanic_Preprocessed.csv”, index=False)

## print(“Preprocessing Completed”)

program 2

---

---

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
```

# Load dataset

```
df = pd.read_csv("twitter_sentiment.csv")
```

# Input and output

```
X = df['text']
y = df['sentiment']
```

# Convert text to numeric

```
vectorizer = CountVectorizer()
X_vectorized = vectorizer.fit_transform(X)
```

# Split data

```
X_train, X_test, y_train, y_test = train_test_split(
X_vectorized,
y,
test_size=0.2,
random_state=42
)
```

# Train KNN model

```
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
```

# Prediction

```
y_pred = model.predict(X_test)
```

# Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print(“Accuracy:”, accuracy)
```

# Report

```
print(classification_report(y_test, y_pred))
```

# program 03

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

# Sample dataset

data = {
‘text’: [
‘I love this movie’,
‘This film is amazing’,
‘Very good experience’,
‘I hate this movie’,
‘Worst film ever’,
‘Very bad acting’
],

```
'sentiment': [
    'Positive',
    'Positive',
    'Positive',
    'Negative',
    'Negative',
    'Negative'
]
```

}

df = pd.DataFrame(data)

# Input and output

```
X = df['text']
y = df['sentiment']
```

# Text to numeric conversion

```
vectorizer = CountVectorizer()
X_vectorized = vectorizer.fit_transform(X)
```

# Split data

```
X_train, X_test, y_train, y_test = train_test_split(
X_vectorized,
y,
test_size=0.2,
random_state=42
)
```

# Train model

```
model = MultinomialNB()
model.fit(X_train, y_train)
```

# Prediction

```
y_pred = model.predict(X_test)
```

# Accuracy

accuracy = accuracy_score(y_test, y_pred)

print(“Accuracy:”, accuracy)

# Test custom sentence

new_text = [“This movie is fantastic”]

new_text_vectorized = vectorizer.transform(new_text)

prediction = model.predict(new_text_vectorized)

## print(“Sentiment:”, prediction[0])

## program 4

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

# Load dataset

df = pd.read_csv(“abalone.csv”)

# Convert categorical data

```
encoder = LabelEncoder()
df[‘Sex’] = encoder.fit_transform(df[‘Sex’])
```

# Input and output

```
X = df.drop(‘Rings’, axis=1)
y = df[‘Rings’]
```

# Split dataset

```
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42
)
```

# Train model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

# Prediction

y_pred = model.predict(X_test)

# Error

mse = mean_squared_error(y_test, y_pred)

print(“Mean Squared Error:”, mse)

# Predict age

age = y_pred + 1.5

print(“Predicted Age:”)
print(age)

## ANN

program1
import numpy as np

# Input value

x = float(input("Enter a value: "))

# Sigmoid Function

sigmoid = 1 / (1 + np.exp(-x))

# ReLU Function

relu = max(0, x)

# Tanh Function

tanh = np.tanh(x)

# Output

print(“Sigmoid :”, sigmoid)
print(“ReLU :”, relu)
print(“Tanh :”, tanh)

## program 2

import numpy as np

# XOR input and output

```
X = np.array([
[0, 0],
[0, 1],
[1, 0],
[1, 1]
])

y = np.array([
[0],
[1],
[1],
[0]
])
```

# Sigmoid activation function

```
def sigmoid(x):
return 1 / (1 + np.exp(-x))
```

# Derivative of sigmoid

```
def sigmoid_derivative(x):
return x * (1 - x)
```

# Random weights

np.random.seed(1)

input_neurons = 2
hidden_neurons = 2
output_neurons = 1

# Initialize weights and bias

hidden_weights = np.random.uniform(size=(input_neurons, hidden_neurons))
hidden_bias = np.random.uniform(size=(1, hidden_neurons))

output_weights = np.random.uniform(size=(hidden_neurons, output_neurons))
output_bias = np.random.uniform(size=(1, output_neurons))

# Learning rate

lr = 0.5

# Training

for epoch in range(10000):

```
# Forward propagation
hidden_input = np.dot(X, hidden_weights) + hidden_bias
hidden_output = sigmoid(hidden_input)

final_input = np.dot(hidden_output, output_weights) + output_bias
predicted_output = sigmoid(final_input)

# Error calculation
error = y - predicted_output
```

```
# Backpropagation
d_predicted_output = error * sigmoid_derivative(predicted_output)

error_hidden_layer = d_predicted_output.dot(output_weights.T)
d_hidden_layer = error_hidden_layer * sigmoid_derivative(hidden_output)

# Updating weights and bias
output_weights += hidden_output.T.dot(d_predicted_output) * lr
output_bias += np.sum(d_predicted_output, axis=0, keepdims=True) * lr

hidden_weights += X.T.dot(d_hidden_layer) * lr
hidden_bias += np.sum(d_hidden_layer, axis=0, keepdims=True) * lr
```

# Final output

print(“Input:”)
print(X)

print("\nActual Output:")
print(y)

print("\nPredicted Output:")
print(predicted_output)

print("\nRounded Output:")
print(np.round(predicted_output))

---

## program 03

---

import numpy as np

# Input data

```
X = np.array([
[0, 0],
[0, 1],
[1, 0],
[1, 1]
])
```

# Output data

```
y = np.array([
[0],
[1],
[1],
[0]
])
```

# Sigmoid activation function

```
def sigmoid(x):
return 1 / (1 + np.exp(-x))
```

# Derivative of sigmoid function

```
def sigmoid_derivative(x):
return x * (1 - x)
```

# Initialize weights

np.random.seed(1)

input_neurons = 2
hidden_neurons = 2
output_neurons = 1

hidden_weights = np.random.uniform(size=(input_neurons, hidden_neurons))
hidden_bias = np.random.uniform(size=(1, hidden_neurons))

output_weights = np.random.uniform(size=(hidden_neurons, output_neurons))
output_bias = np.random.uniform(size=(1, output_neurons))

# Learning rate

learning_rate = 0.5

# Training process

for epoch in range(10000):

```
# Feed Forward
hidden_input = np.dot(X, hidden_weights) + hidden_bias
hidden_output = sigmoid(hidden_input)

final_input = np.dot(hidden_output, output_weights) + output_bias
predicted_output = sigmoid(final_input)

# Error
error = y - predicted_output

# Backpropagation
d_output = error * sigmoid_derivative(predicted_output)
```

```
hidden_error = d_output.dot(output_weights.T)
d_hidden = hidden_error * sigmoid_derivative(hidden_output)

# Update weights and biases
output_weights += hidden_output.T.dot(d_output) * learning_rate
output_bias += np.sum(d_output, axis=0, keepdims=True) * learning_rate

hidden_weights += X.T.dot(d_hidden) * learning_rate
hidden_bias += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate
```

# Output

print(“Input Data:”)
print(X)

print("\nActual Output:")
print(y)

print("\nPredicted Output:")
print(predicted_output)

print("\nRounded Output:")
print(np.round(predicted_output))

---

---

## program 04

---

import numpy as np
import matplotlib.pyplot as plt

# Input data for AND gate

```
X = np.array([
[0, 0],
[0, 1],
[1, 0],
[1, 1]
])
```

# Output for AND gate

```
y = np.array([0, 0, 0, 1])
```

# Initialize weights and bias

```
weights = np.zeros(2)
bias = 0
learning_rate = 0.1
epochs = 10
```

# Activation function

```
def step_function(x):
if x >= 0:
return 1
else:
return 0
```

# Perceptron learning

for epoch in range(epochs):
for i in range(len(X)):
linear_output = np.dot(X[i], weights) + bias
prediction = step_function(linear_output)

```
    error = y[i] - prediction

    weights = weights + learning_rate * error * X[i]
    bias = bias + learning_rate * error
```

print(“Final Weights:”, weights)
print(“Final Bias:”, bias)

# Plot data points

for i in range(len(X)):
if y[i] == 0:
plt.scatter(X[i][0], X[i][1], marker=‘o’, label=‘Class 0’ if i == 0 else “”)
else:
plt.scatter(X[i][0], X[i][1], marker=‘x’, s=100, label=‘Class 1’)

# Decision boundary

x_values = np.linspace(-0.5, 1.5, 100)

# Formula: w1*x + w2*y + b = 0

y_values = -(weights[0] * x_values + bias) / weights[1]

plt.plot(x_values, y_values, label=‘Decision Boundary’)

```
plt.xlabel(“Input 1”)
plt.ylabel(“Input 2”)
plt.title(“Perceptron Decision Region for AND Gate”)
plt.legend()
plt.grid(True)
plt.show()
```